Mastering Jenkins Pipelines and Administration

starts at 9:05 pm

Agenda

1. Build Triggers
    1. Types of Build Triggers
2. Parallel Pipeline
3. Multibranch Pipeline
4. Folder
5. Notifications in Jenkins
6. Reports

→ Build Triggers

(1) Manual Pressing.
(2) Schedule it.
(3) Trigger it remotely
(4) Dependency based
(add water, milk, coffee)

→ Job, pipeline

→ Why ?

① Save time

② Reduces human error

③ Ensures CI/CD

2025.01.M

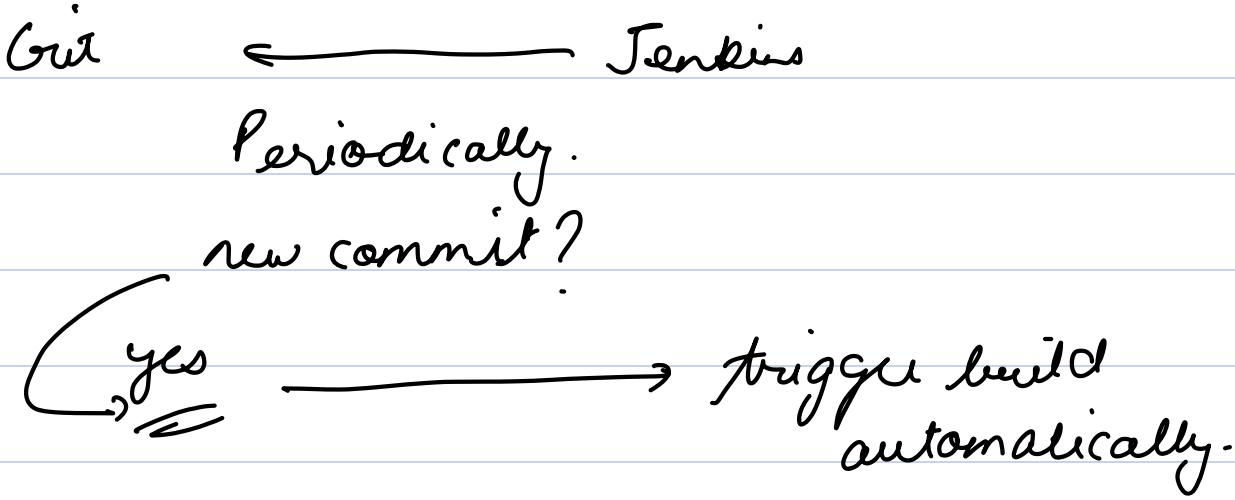
2024.01.M

?

→ Types of triggers

① Manual trigger.

② SCM Polling.



① Create a new Job

② Configure git repository in Jenkins.

(3) Enable Poll SCM

(4) Add a Build Step

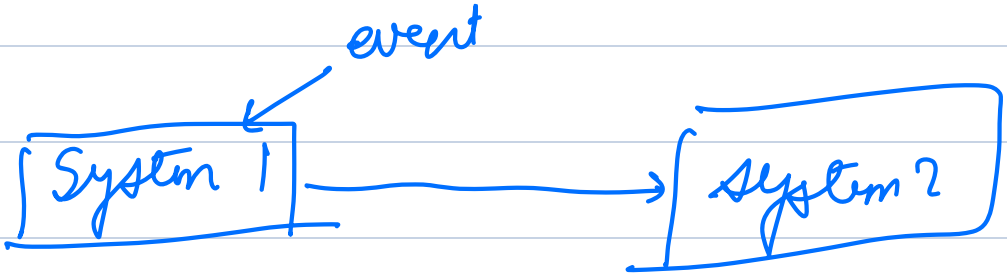
(5) Test SCM Polling

add new file (github)

↓

Build will be triggered automatically.

(3) Webhook Trigger.



Why use webhooks?

Instant Builds – Webhooks notify Jenkins immediately, unlike polling which waits for a scheduled check.

Efficient – Reduces unnecessary server load caused by frequent polling.

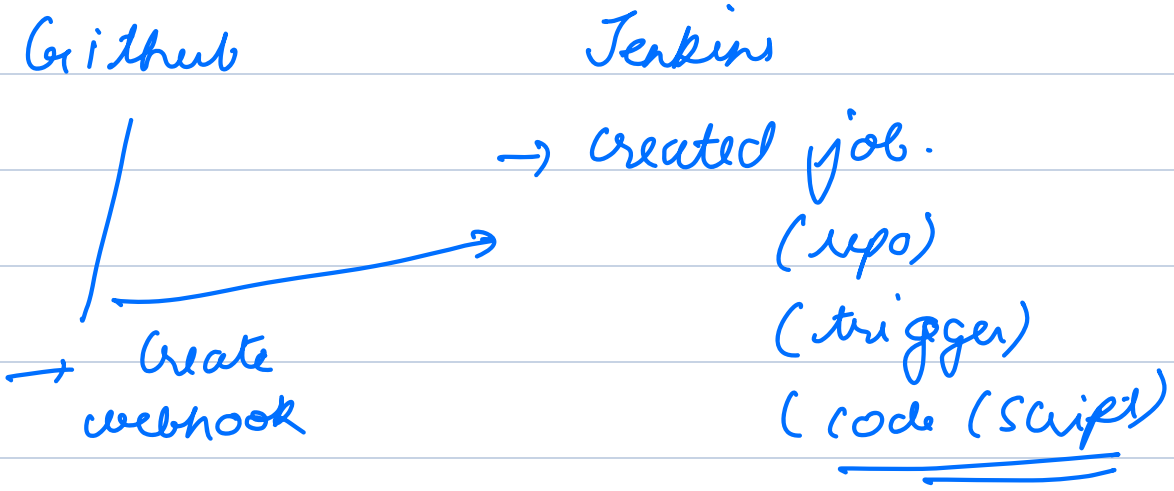
Better CI/CD – Faster feedback loop for developers.

→ Demo webhooks.

(1) Install plugin

① github plugin
② github integration plugin

② Create Job (Jenkins) and webhook (github)



③ Test

④ Scheduled Build

⑤ github Branches

testing-repo
→ jenkins (Jenkinsfile)
→ main "
→ x "
→ y "
→ z

⑥ Github Pull Requests

⑦ Trigger Builds Remotely

HTTP request.
→ API token

Demo
→ ① Create Pipeline and choose Token name (Trigger)
② Create Token (Profile → Security)
③ Create Jenkinsfile.
④ Trigger Build using curl.

```
curl -X POST "http://<JENKINS_SERVER_IP>:8080/job/GitHub-Pipeline-Demo/build?token=my-secret-token" \
  --user "your-username:your-api-token"
```

Break → 10:20 pm

→ Notifications

Configure SMTP Server in Jenkins

1. Go to **Manage Jenkins** → **Configure System**.

2. Scroll to **E-mail Notification**.

3. Set the SMTP Server (e.g., for Gmail: `smtp.gmail.com`).

4. Enable **Use SMTP Authentication** and provide:

   - **Username:** Your Gmail address.

   - **Password:** Use an **App Password** (not your regular Gmail password).

5. Click **Test configuration** → Send a test email.

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Building project...'
            }
        }
        stage('Test') {
            steps {
```

```
            echo 'Running tests...'
        }
    }
  }

  post {
    success {
        emailext subject: "Jenkins Job Successful: ${env.JOB_NAME}",
                 body: "The job ${env.JOB_NAME} (#${env.BUILD_NUMBER}) has completed successfully.\nCheck it here: ${env.BUILD_URL}",
                 to: 'developer@example.com'
    }
    failure {
        emailext subject: "Jenkins Job Failed: ${env.JOB_NAME}",
                 body: "The job ${env.JOB_NAME} (#${env.BUILD_NUMBER}) has failed.\nCheck logs: ${env.BUILD_URL}",
                 to: 'developer@example.com'
    }
  }
}
```

→ Parallel Pipelines

A **Parallel Pipeline** in Jenkins allows multiple stages to execute **simultaneously** rather than sequentially.

Stage 1 → 10 mins
Stage 2 → 5 min
Stage 3 → 15 min
Stage 4 → 15 min.

3 & 4 → no dependency.
1 & 2 → need to run 1 after other.

Most efficient
1 & 2 Sequentially.
3 & 4 Parallely.
10 + 5 + 15 → 30 mins.

Why?
① speeds up execution
② efficient resource usage
③ Better pipeline structure.

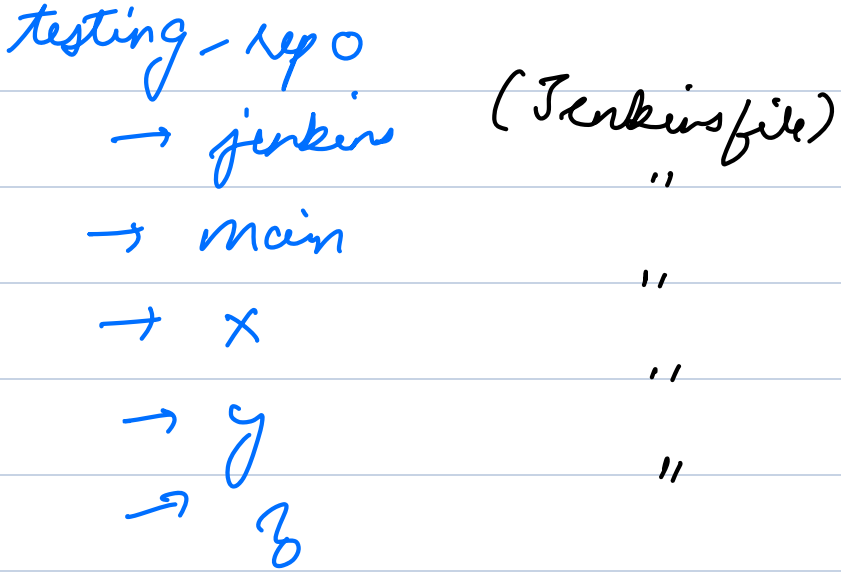
```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo "Building the application..."
                sh 'sleep 15'
            }
        }

        stage('Parallel Execution') {
            parallel {
                stage('Test') {
                    steps {
                        echo "Running tests..."
                        sh 'sleep 15'
                    }
                }
                stage('Deploy') {
                    steps {
```

```
                echo "Deploying application..."
                sh 'sleep 15'
            }
        }
    }
  }
}
```

→ Multi Branch Pipeline



A **Multibranch Pipeline** in Jenkins is used when a repository has multiple branches, and you want Jenkins to **automatically detect and create pipelines** for each branch that contains a `Jenkinsfile`.

Key Features

1. **Auto-Discovery:** Jenkins automatically discovers branches with a `Jenkinsfile` and sets up pipelines.

2. **Automatic Triggering:** Pipelines run automatically when changes are pushed to the respective branches.

3. **Branch-Specific Pipelines:** Each branch can have its own CI/CD logic.

4. **Support for Pull Requests:** Jenkins can trigger builds on pull request creation and updates.

Demo → Multibranch Pipeline.

1. main
2. jenkins
3. feature - 1
4. bugfix - 1

(1) Create Branches. and create Jenkinsfile in all 4 branches.

(2) Create a multibranch pipeline

→ GitHub

→ API request limits

→ 60 min.

→ Folders.

Project Based Organisation

├── Project_A/

| ├── Build_Job

| ├── Deploy_Job

| ├── Test_Job

├── Project_B/

| ├── Build_Job

| ├── Deploy_Job

Environment Based Organisation

├── Dev/

| ├── App_Build

| ├── App_Deploy

├── Prod/

| ├── App_Build

| ├── App_Deploy